Problem 1: 235
Problem 2: 215
Problem 3: 250
Problem 4: 235

Date: .... No: ....

Name: Trương Quang Trực
Class: ~~AE~~ E25 CQCECU-N
MSSV: N25DECE074

235

Problem 1: Singly linked list - In-place Reversal and Rewire Tracing

Given a singly linked list head → [10] → [85] → [40] → [15] → NULL

- Task 1: Write a function (algorithm in pseudocode) to reverse the singly linked list, including three pointers (prev, curr, next) to complete your function
- Task 2: Draw a diagram or step-by-step pointer trace table showing the positions and values of the three pointers (prev, curr, next) after each execution until the list is fully reversed.

- Solve:

1.1

- Task 1:

```
Function Reverse (head):
    prev ← NULL
    curr ← head
    next ← NULL
    while (curr != NULL)
        next ← curr.next
        curr.next ← prev
        prev ← curr
        curr ← next.
    return prev
```

Thành Đạt

Scanned with CamScanner

Date: .... No: ....

1.25 - Task 2:

Execution	prev	curr	next	list so far
Start:	NULL	10	-	~~10 →~~
1	10	25	25	10 → NULL
2	25	40	40	25 → 10 → NULL
3	40	15	15	40 → 25 → 10 → NULL
4	15	NULL	NULL	15 → 40 → 25 → 10 → NULL

2.5 Problem 2: Circular linked list:

Given a singly circular linked list: head → [10] → [20] → [30] → (points back to head)

- Task 1: Write pseudocode for function. InsertHead (head, newValue) to insert a new node containing newValue at the ~~front~~ front of the circular linked list while maintaining its circular structure.

- Task 2: Trace the execution steps (use next description or pointer diagrams) when inserting newValue = 5 into the given list.

⊙ Solve:

1.25 - Task 2: Trace with newValue = 5 on 10 → 20 → 30 → (back to 10).

- First: Create node 5.
- 2: walk from 10 is: 10 → 20 → 30. Since 30.next == head, tail = 30.
- 3: 5.next = 10.
- 4: 30.next = 5.
- 5: head = 5.

Scanned with CamScanner

Date: .... No: ....

=> The final result is: 5 → 10 → 20 → 30 → (back to 5)

1.25 - Task 1:

```
function InsertHead(head, newValue):
    node ← new Node(newValue)
    if head == NULL:
        node.next ← node
        return node
    tail ← head
    while tail.next != head:
        tail = tail.next
    node.next = head
    tail.next = node
    return node
```

Problem 5: Doubly linked list:

Given a doubly linked list containing integers.

- Task 1: Write pseudocode for InsertBefore(head, target, newValue) to insert a new node containing newValue immediately before the first occurrence of target
- Task 2: List the exact order of pointer assignment required and briefly justify why this specific sequence avoids memory leaks or broken links
- Task 3: Explicitly handle the following two edge case in code / pseudocode:
  1. Target is located at the head node (head)
  2. Target does not exist in the list

Thành Đạt

Scanned with CamScanner

Date: .... No: ....

0.75 - Task 1:

```
function InsertBefore(head, target, newValue)
    cur = head
    while (cur != NULL and cur.data != target)
        cur = cur.next
    if cur == NULL
        return head

    node = new Node(newValue)
    node.prev = cur.prev
    node.next = cur
    if (cur.prev != NULL)
        cur.prev.next = node
    else
        head = node
    cur.prev = node
    return head
```

0.75 - Task 2: The general case: A ↔ B ↔ C, inserting N before B. (so cur = B).

We have: Before: A ↔ B ↔ C

Step 1: N.prev = B.prev (N points back to A)

Step 2: N.next = B (N points back to B).

Step 3: A.next = N (written as B.prev.next = N)

Step 4: B.prev = N (B points back to N)

After: A ↔ N ↔ B ↔ C

Scanned with CamScanner

Date: .... No: ....

Name: Tường Quang Tuấn.
Class: ELS ECE01-N
MSSV: NELSECE044

- Task 5 (Problem 5): The two edge case.

with Edge case 1: The target is the head.

~~Example~~ Before: NULL ⇔ [1] ⇔ [2], target = 1, insert = 0

- Step 1: N.prev = NULL
- Step 2: N.next = [1]
- Step 3: Skipped, ~~head~~ head = N (N became the new head)
- Step 4: [1].prev = N.
- After: NULL ⇔ [0] ⇔ [1] ⇔ [2].

1,0 • Task 5 (Problem 5): The two edge case:

with Edge case 1: The target is the head:

- ~~Problem code~~ We have code in C++:

```
Dnode* node = new Dnode(newValue);
node->prev = cur->prev;
node->next = cur;
if (cur->prev != nullptr)
    cur->prev->next = node;
else
    head = node;
cur->prev = node;
return head;
```

Thành Đạt

Scanned with CamScanner

Date: .... No: ....

and the Trace is: list have 1 ↔ 2 ↔ 3, call InsertBefore(head, 1, 0)

- Before: head → [1] ↔ [2] ↔ [3]
  cur = [1], cur->prev = NULL.
- Step 1: N->prev = cur->prev = NULL (N points back to nothing)
- Step 2: N->next = [1] (N points forward to the old head)
- Step 3: cur->prev is NULL, so head = N (the head pointer moves to N)
- Step 4: [1]->prev = N (the old head points back to N)

After: head → [0] ↔ [1] ↔ [2] ↔ [3]

- Edge case 2: handle with code:

```
DNode* cur = head;
while (cur != nullptr && cur->data != target)
    cur = cur->next;
if (cur == nullptr)
    return head;
```

And we have a trace to explain:

loss: 1 ↔ 2 ↔ 3, call InsertBefore(head, 42, 7).

Step	cur	cur->data != 42	Action
1	1	true	cur = cur->next
2	2	true	cur = cur->next
3	3	true	cur = cur->next
4	NULL	Fails	Exit loop.

Date: .... No: ....

2.5 Problem 4: Sorted linked list:

Given a singly linked list sorted in ascending order.

head → [2] → [5] → [8] → NULL

* Task 1: Write the function SortedInsert(head, val) to insert into the list such that the ascending order is preserved.

- Task 2 Construct a step-by-step trace table detailing pointer interactions and condition evaluation when inserting val = 6 into the list.

Solve:

1.25 Task 1:

```
function SortedInsert(head, val):
    node = new Node(val)
    if (head == NULL or val < head.data):
        node.next = head
        return node
    curr = head
    while (curr.next != NULL and curr.next.data < val):
        curr = curr.next
    node.next = curr.next
    curr.next = node
    return head
```

1.25 Task 2: Trace

Thành Đạt

Scanned with CamScanner

Date: .... No: ....

Inserting val = 6 into 2 → 5 → 8.

Step	curr	curr.next	Condition	Action
Pre-check	-	-	head is not NULL, and 6 ≤ 2 (false)	skip
1	2	5	true (5<6)	curr = 5
2	5	8	false (8<6)	stop
Finish	5	8	-	node.next = 8; then 5.next = 6

The Result is: 2 → 5 → 6 → 8 → NULL